



2. Threads

Celso Paím



Universidade Católica de Angola

Faculdade de Engenharia Informática

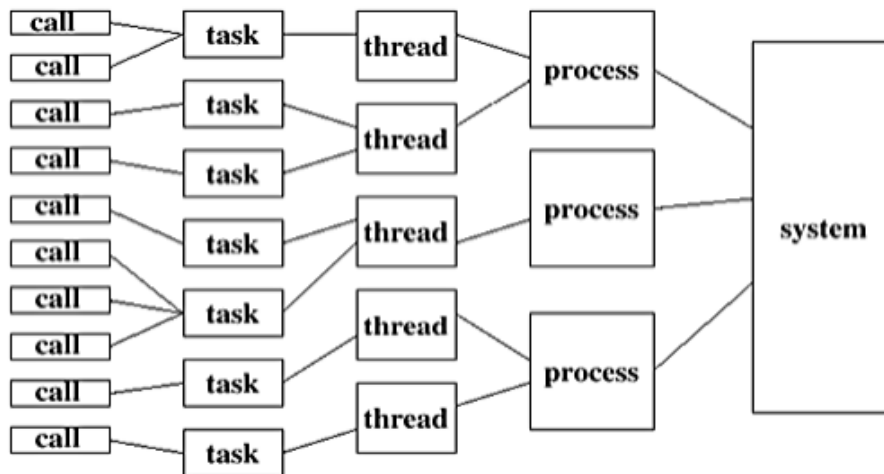
Sistemas Distribuídos e Paralelos I

2.1. Concorrência e Paralelismo

- Programação concorrente ou programação simultânea é um paradigma de programação para a construção de programas que fazem uso da execução simultânea de várias tarefas interactivas, que podem ser implementadas como programas separados ou como um conjunto de *threads* criadas por um único programa.
- **Concorrência** refere-se à capacidade de um sistema de lidar com múltiplas tarefas ao mesmo tempo. Não significa necessariamente que essas tarefas são executadas simultaneamente, mas sim que o sistema pode alternar rapidamente entre as tarefas, de modo que parece que estão a ser processadas em simultâneo. Em Java, a concorrência pode ser feita usando *threads*. Exemplo em web services, processamento de I/O, aplicações baseadas em GUI...
- **Paralelismo** refere-se à execução simultânea de várias tarefas. Isso requer múltiplos núcleos de CPU, onde cada núcleo pode executar uma tarefa diferente ao mesmo tempo (o paralelismo é sobre fazer várias realente em simultâneo). Em Java, o paralelismo é bem representado por APIs como o *ForkJoinPool* ou o uso de *streams*.



2.1. Concorrência e Paralelismo



- Uma aplicação Java pode ser:
 - Nem concorrente e nem paralela (tipico programa com uma única *thread* que executa sequencialmente cada comando até seu final);
 - Concorrente, mas não paralela;
 - Paralela, mas não concorrente;
 - Concorrente e paralela.



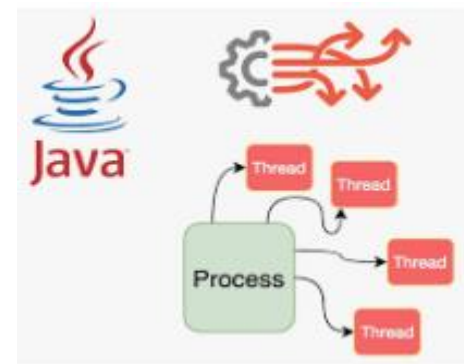
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2 Threads

- Toda aplicação Java executa implicitamente a *thread main* criada pela JVM.
- Outras *threads* internas em segundo plano também podem ser iniciadas durante a inicialização da JVM. O número e a natureza dessas *threads* variam entre as implementações da JVM. No entanto, todas as *threads* de nível do utilizador são explicitamente construídas e iniciadas a partir da *thread* principal, ou de quaisquer outras *threads* que elas, por sua vez, criem.





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

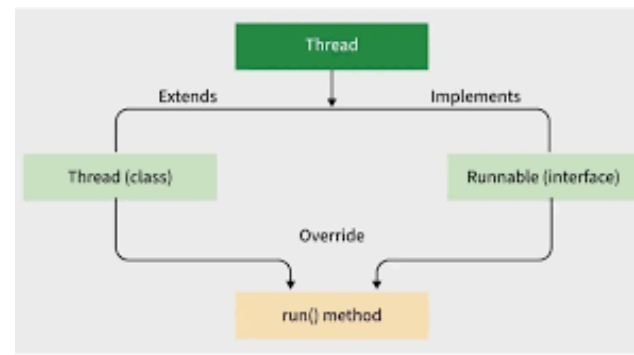
2.2 Threads

O que é então uma thread?

- *Threads* são segmentos de execução independente de um programa, podendo ser executadas simultaneamente, ou de forma assíncrona ou síncrona; compartilhando recursos subjacentes do sistema, como ficheiros, bem como acesso a outros objectos construídos dentro do mesmo programa.
- Uma *threads* em Java pode ser criada extendendo a classe `Thread` ou implementando a interface `Runnable` e devem sempre reescrever o método `run()`.

<https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>

<https://docs.oracle.com/javase/8/docs/api/java/lang/Runnable.html>





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

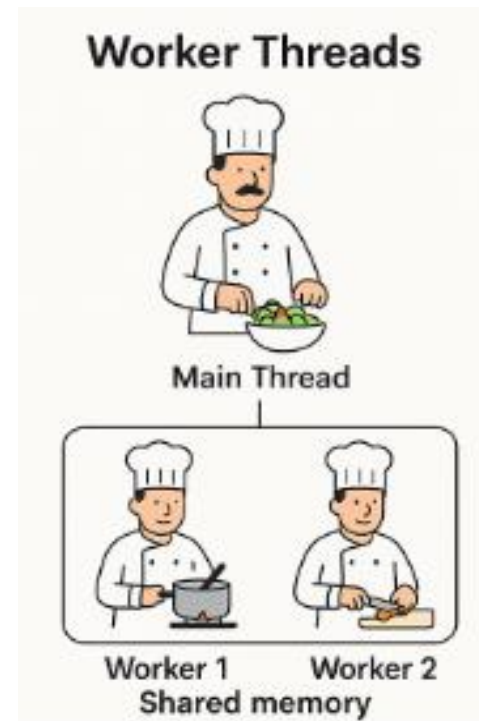
2.2 Threads

	extends Thread	implements Runnable
Herança	Limita a classe. Não permite outra herança.	Permite estender outras classes.
Flexibilidade	Baixa.	Alta.
Design	Acoplado (Tarefa + Thread).	Desacoplado (Separação de tarefas).
Reusabilidade	Baixa. Cada thread criada é um objeto único. Compartilhar recursos entre threads requer mais sincronização.	Alta (fácil de usar com pools). O mesmo objeto Runnable pode ser passado para múltiplos objectos Thread facilitando o compartilhamento de recursos e variáveis entre threads
Recomendação	<i>Tarefas simples e rápidas.</i>	<i>Sistemas reais, robustos e modulares.</i>

2.2 Threads

Por exemplo, no streaming de áudio ou vídeo pela internet, talvez o utilizador não queira esperar até que todo o áudio ou vídeo seja baixado antes de iniciar a reprodução. Para resolver esse problema, múltiplas *threads* podem ser usadas — uma para baixar o áudio ou vídeo (que pode ser classificada como ***thread produtora***) e outra para reproduzi-lo (pode ser classificada como ***thread consumidora***).

Essas actividades prosseguem concorrentemente. Para evitar reprodução instável, as *threads* são **sincronizadas** (isto é, suas ações são coordenadas) para que a *thread* do *player* só comece depois que houver uma quantidade suficiente de áudio ou vídeo na memória para manter a *thread* do *player* ocupada. *Threads* produtoras e consumidoras compartilham a memória.





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2 Threads (remarks)

- Em java as threads estão no pacote `java.lang` e mantêm um registo/controla da sua actividade.
- As *threads* podem ser sincronizadas ou não. Java define a palavra-chave `synchronized` e os métodos `wait()`, `notify()` e `notifyAll()` que todos objectos herdam da superclasse `Object`.
- As *threads* podem ser classificadas como produtoras ou consumidoras.
- As *threads* pode ter uma prioridade que vai de 0 a 10.
- Uma *thread* pode ter vários estados.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2 Threads (remarks)

- Principais métodos próprios de uma *thread*:

Método	Propósito
start()	Inicia a <i>thread</i> . Cria uma nova pilha de execução e chama o <code>run()</code>
run()	Contém o código que a <i>thread</i> vai executar
sleep(long ms)	Pausa a <i>thread</i> actual por X ms
join()	A <i>thread</i> atual espera essa <i>thread</i> terminar
currentThread()	Retorna a referência da <i>thread</i> que está executando agora
getName() / setName(String n)	getter/setter do nome da <i>thread</i>
getId()	Retorna o ID único da <i>thread</i>
isAlive()	Retorna <code>true</code> se a <i>thread</i> foi iniciada e ainda não morreu
getState()	Retorna o estado: <code>NEW</code> , <code>RUNNABLE</code> , <code>BLOCKED</code> , <code>WAITING</code> , <code>TIMED_WAITING</code> , <code>TERMINATED</code>
interrupt()	Manda um sinal de interrupção mas não mata a <i>thread</i>
getPriority() / setPriority(int)	getter/setter da prioridade da <i>thread</i> (de 1 a 10. <code>MIN_PRIORITY=1</code> , <code>NORM_PRIORITY=5</code> , <code>MAX_PRIORITY=10</code>)
setDaemon(boolean)	Marca a <i>thread</i> como <i>daemon</i> (ou <i>thread</i> de apoio)
stop()	Mata a <i>thread</i> mas pode deixar objectos em estado inconsistente
suspend()	Pausa a <i>thread</i> mas liberar locks, podendo causar deadlock
resume()	Retomar a execução da <i>thread</i>



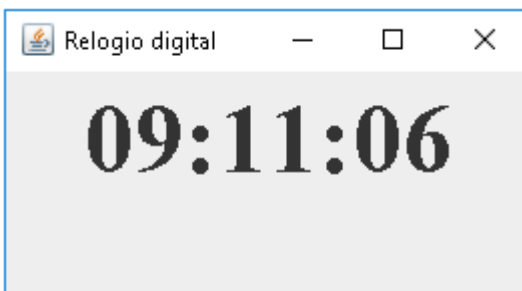
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

Exercicio Thread com `extends` – Relógio Digital

- Exemplo de uma aplicação concorrente, mas não paralela com recurso a uma *thread* que actualiza a cada segundo a hora, implementando um relógio digital numa JFrame;





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
1.  /*
2.  Autor: Celso Paim
3.  Data: Novembro/2025
4.  Prog_177: Implementar um relógio digital com uma thread
5.  */
6.  package edu.livrofundamentos.exemplos.cap6;
7.
8.  import java.awt.*;
9.  import java.text.SimpleDateFormat;
10. import java.util.*;
11. import javax.swing.*;
12.
13. public class RelogioDigital extends JFrame {
14.     private JLabel horaLabel;
15.
16.     public RelogioDigital() {
17.         super( "Relógio digital" );
18.         Container container = getContentPane();
19.         container.setLayout( new FlowLayout() );
20.         horaLabel = new JLabel();
21.         horaLabel.setFont( new Font( "Times New Roman", Font.BOLD, 50 ) );
22.         container.add( horaLabel );
23.         setSize( 240, 150 );
24.         setVisible( true );
25.     }
26.
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
27. public void setHora( String str ) { horaLabel.setText( str ); }
28.
29. public static void main( String[] args ) {
30.     RelogioDigital relógio = new RelogioDigital();
31.     new Thread() {
32.         @override
32.         public void run() {
33.             try {
34.                 while( true ) {
35.                     Calendar cal = Calendar.getInstance();
36.                     SimpleDateFormat formatter = new SimpleDateFormat( "hh:mm:ss" );
37.                     Date date = cal.getTime();
38.                     String horaStr = formatter.format( date );
39.                     relógio.setHora( horaStr );
40.                     this.sleep( 1000 );
41.                 }
42.             }
43.             catch ( Exception ex ) { }
44.         }
45.     }.start();
46. }
47. }
```



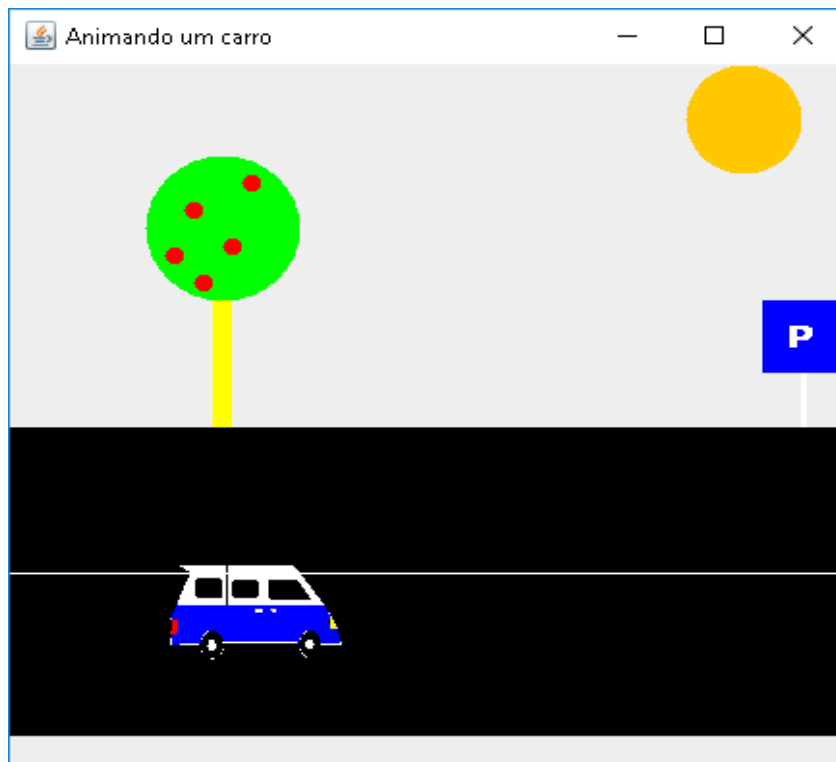
Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

Exercicio Thread com `implements Runnable` – Animação

- Exemplo de uma aplicação concorrente, mas não paralela com recurso a uma *thread* que simula um carro a se deslocar pela tela;





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
1.  /*
2.  Autor: Celso Paim
3.  Data: Novembro/2025
4.  Prog_178: Simular a movimentacao de uma imagem na tela com uma thread
5.  */
6.  package edu.livrofundamentos.exemplos.cap6;
7.
8.  import java.awt.*;
9.  import javax.swing.*;
10.
11.  public class MovimentarCarro extends JPanel implements Runnable {
12.      private ImageIcon picture, parkSimbol;
13.      private int x, y;
14.      private static final int WIDTH = 450, HEIGHT = 450;
15.
16.      Thread moveThread = null;
17.
18.      public MovimentarCarro() {
19.          inicializarAnimacao();
20.      }
21.
22.      protected final void inicializarAnimacao() {
23.          picture = new ImageIcon( "carrinho" + getNumCarro() + ".gif" );
24.          parkSimbol = new ImageIcon( "sinal_parque.gif" );
25.          x = 0;
26.          y = HEIGHT - 210;
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
}
```

```
public int getNumCarro() {  
    int num = 1 + (int)( Math.random() * 7 );  
    return num;  
}
```

@Override

```
public void paintComponent( Graphics g ) {  
    super.paintComponent( g );
```

```
    // desenhar um rectangulo para a estrada
```

```
    g.setColor( Color.black );  
    g.fillRect( 0, HEIGHT - 250, WIDTH, HEIGHT - 280 );
```

```
    // desenhar a imagem do carro
```

```
    g.drawImage( picture.getImage(), x, y, WIDTH - 320, HEIGHT - 320, null );
```

```
    // desenhar uma arvore
```

```
    g.setColor( Color.yellow );  
    g.fillRect( WIDTH - 345, HEIGHT - 320, WIDTH - 440, HEIGHT - 380 ); // arvore  
    g.setColor( Color.green );  
    g.fillOval( WIDTH - 380, HEIGHT - 400, WIDTH - 370, HEIGHT - 370 ); // copa da arvore  
    g.setColor( Color.red );  
    g.fillOval( WIDTH - 330, HEIGHT - 390, 10, 10 ); // frutas na arvore
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
g.fillOval( WIDTH - 340, HEIGHT - 355, 10, 10 );  
g.fillOval( WIDTH - 355, HEIGHT - 335, 10, 10 );  
g.fillOval( WIDTH - 360, HEIGHT - 375, 10, 10 );  
g.fillOval( WIDTH - 370, HEIGHT - 350, 10, 10 );
```

// desenhar o pe do sinal de transito

```
g.setColor( Color.white );  
g.fillRect( WIDTH - 40, HEIGHT - 280, 3, 30 );  
g.drawLine( 0, 280, WIDTH, 280 );
```

// desenhar a imagem do sinal de estacionamento

```
g.drawImage( parkSimbol.getImage(), WIDTH - 60, HEIGHT - 320, 40, 40, null );
```

// desenhar o sol

```
g.setColor( Color.orange );  
g.fillArc( WIDTH - 100, 0, 60, 60, 180, 360 );
```

```
}
```

```
public void executarAnimacao() {  
    if( moveThread == null ) {  
        moveThread = new Thread( this );  
        moveThread.start();  
    }  
}
```




Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

@Override

```
public void run() {  
    while( true ) {  
        x += 5;  
        if( x == WIDTH - 40 ) {  
            x = 0;  
            inicializarAnimacao();  
        }  
        repaint();  
        try { moveThread.sleep( 165 ); }  
        catch( InterruptedException ex ) { }  
    }  
}
```

```
public static void main( String[] args ) {  
    MovimentarCarro painel = new MovimentarCarro();  
    JFrame window = new JFrame( "Animando um carro" );  
    Container container = window.getContentPane();  
    container.add( painel );  
    window.pack();  
    window.setSize( WIDTH, HEIGHT );  
    window.setVisible( true );  
    painel.executarAnimacao();  
}
```